

BrandBridge AI

Technical Architecture, Database, API & Implementation Guide

IIT Bombay AI / Data Science Capstone | Team implementation baseline

Purpose

A single source of truth for the five-person team: requirements, architecture, database schema, API contracts, AI/ML design, social integrations, security, Docker/cloud path, testing, work ownership and implementation sequence.

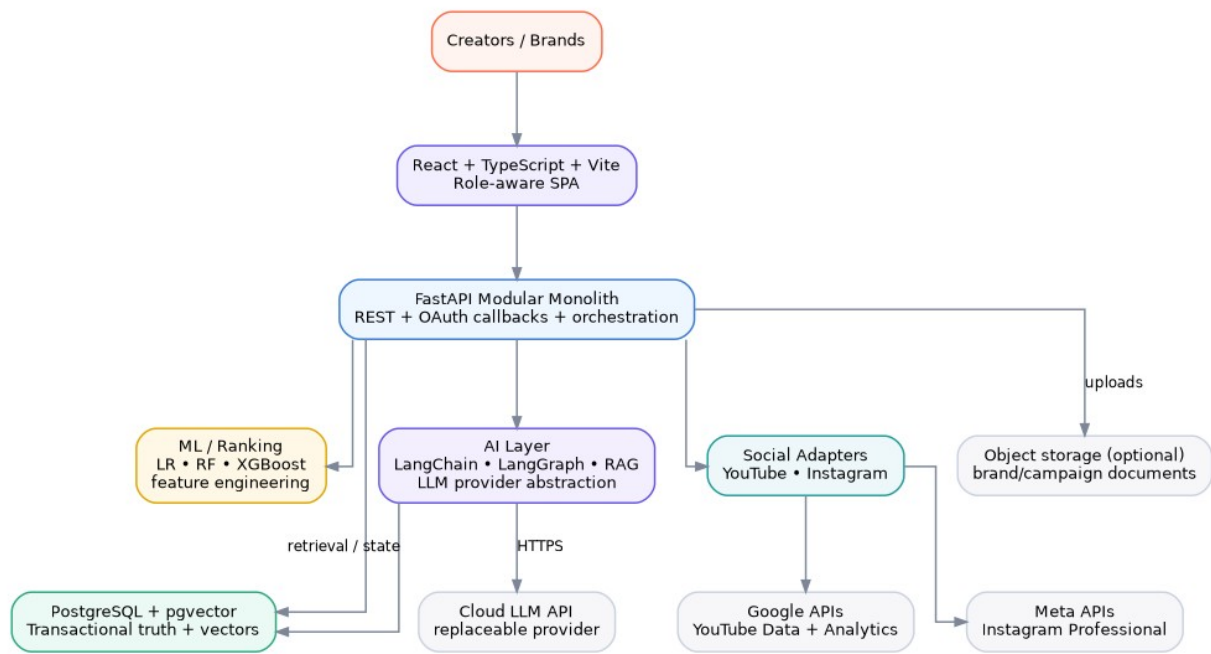


Figure 1. BrandBridge AI end-to-end architecture.
Version 1.0 - consolidated 20 September 2026

Document Control & Source Precedence

Source / decision baseline	How this guide uses it
BrandBridge AI Consolidated Project Plan (V3)	Primary research baseline: reciprocal matching, hard constraints, retrieval/ranking, fairness, evaluation,

Source / decision baseline	How this guide uses it
	MVP/stretch boundaries.
Creator Profile and Data Strategy	Creator evidence model, authorized social data, multi-view representation, dataset scale and gold-standard set.
BrandBridge AI Business / Academic Structure	Problem framing, two-sided product model, hybrid intelligence stack and capstone positioning.
BrandBridge AI UI Wireframes	Provisional client-side baseline: one account/two modes, wall/discovery, campaign studio, role-aware proposals, agreement and collaboration tracking.
Phase 1 Engineering Foundation + prior technical design	Monorepo, modular monolith, FastAPI/PostgreSQL/pgvector, Docker, testing, CI, LangChain/LangGraph boundaries and code conventions.
Current team implementation decisions in chat	BrandBridge AI naming, YouTube and Instagram POCs, Instagram Facebook-login path for professional account insights, UI architecture parked for later revision.

Precedence rule

If sources differ, this guide adopts the latest research scope first, then the Phase 1 engineering conventions, then the current implementation decision. Earlier “CreatorConnect AI” references are treated as the same predecessor concept and renamed BrandBridge AI.

Implementation Map

Part	What the team should use it for
A. Product & Requirements	Freeze scope and understand the end-to-end user/business problem.
B. Software Architecture	Build the monorepo, frontend/backend boundaries and deployment model.
C. Database & APIs	Implement shared contracts without each member inventing their own schema.
D. Social Integrations	Complete YouTube/Instagram OAuth POCs and normalize creator metrics.
E. Intelligence Layer	Implement deterministic filters, retrieval, ranking, RAG and LangGraph in the right order.
F. Research & Evaluation	Create defensible labels, experiments, metrics, fairness/cold-start analysis.
G. Delivery & Governance	Git workflow, CI, security, ownership, milestones and definition of done.

A. Product Definition and Requirements

1. Project definition

BrandBridge AI is an explainable, reciprocal creator-brand recommendation and collaboration system. The product layer is a two-sided marketplace; the capstone contribution is the retrieval/ranking experiment and grounded, human-controlled AI support around it.

Working research question

Can an explainable reciprocal recommender preserve brand relevance while improving mutual creator-brand suitability and opportunity exposure compared with conventional one-sided ranking?

2. Core product principles

- One account can activate Creator, Brand, or both professional profiles.
- Both sides can initiate: Brand → Creator invitation and Creator → Brand proposal are first-class flows.
- Hard commercial/safety constraints are deterministic and evaluated before ranking.
- AI recommendations expose component scores and evidence instead of only a final percentage.
- Commercial actions remain human-controlled; AI may draft, recommend or explain but never self-commit.
- Social APIs enrich and verify profiles but never become a mandatory dependency for the base product.
- RAG is reserved for unstructured evidence such as brand guidelines and campaign PDFs, not SQL facts.
- The system must remain demoable using seeded/manual creator data when external APIs are unavailable.

3. Target personas and required capabilities

Persona	Required MVP capabilities
Brand / SME marketing manager	Create brand profile; create campaign; parse free-text brief; confirm structured constraints; discover ranked creators; inspect reasons; shortlist/invite; review proposals; track collaboration.
Micro/nano creator	Maintain profile/preferences/rate card; optionally connect social accounts; discover suitable campaigns; inspect match reasons; submit proposal; review invitations.
Growing creator	Demonstrate niche/audience/content fit beyond raw follower count using structured metrics and semantic representation.
Admin / research team	Seed data; manage labels/model versions; inspect score components; run evaluation and demo scenarios.

4. Functional requirements

ID	Area	Requirement
FR-01	Identity	Register/login; one identity; activate

ID	Area	Requirement
		Creator and/or Brand profile.
FR-02	Creator profile	Bio, location, languages, categories, preferences, exclusions, availability, audience fields.
FR-03	Rate card	Platform/content-type rates with currency and effective status.
FR-04	Brand profile	Industry, products/services, values/exclusions, website and knowledge assets.
FR-05	Campaign	Raw brief + structured requirements + objective + budget + target audience + hard constraints.
FR-06	Campaign parsing	LLM proposes structured fields; brand reviews/edits before matching.
FR-07	Discovery	Brand discovers creators; creator discovers campaigns; filters and ranked results.
FR-08	Explainability	Show structured, semantic, reciprocal and rerank components plus evidence/freshness.
FR-09	Proposal	Creator proposal and Brand invitation; transparent deliverables, price and message.
FR-10	Collaboration	Accepted proposal creates collaboration; basic status tracking and optional messages.
FR-11	Documents/RAG	Upload campaign/brand docs; retrieve relevant chunks for grounded Q&A/proposal checks.
FR-12	Social verification	Optional YouTube and Instagram professional-account connection; normalized metrics snapshots.
FR-13	Research labels	Store pair labels/rater rationale and model/feature versions.
FR-14	Audit	Record AI runs, prompt/model versions, latency, status and errors.
FR-15	Fallback	Manual/seed creator data supports all core flows without external APIs.

5. Non-functional requirements

ID	Requirement	Acceptance target
NFR-01	Security	Secrets never committed; OAuth tokens server-side and encrypted/token-referenced; least-privilege scopes.
NFR-02	Reproducibility	docker compose up --build starts frontend, backend and PostgreSQL/pgvector.
NFR-03	Explainability	Every recommendation keeps score components, feature/model versions and metric freshness.
NFR-04	Testability	Routes→services→repositories; AI/ML functions independently testable; notebooks not production code.
NFR-05	Resilience	External API failure does not block manual profile/search/recommendation demo.
NFR-06	Performance	Interactive API target generally <2s without LLM; AI flows log latency and may run asynchronously if needed.
NFR-07	Privacy	Minimize PII; no sensitive-trait inference; show missingness and freshness.
NFR-08	Research rigor	Held-out splits, versioned datasets/prompts/models, no unsupported conversion/sales claims.

6. MVP versus stretch

Core MVP - must work end to end	Stretch - only after core evaluation works
Brand + creator profiles/preferences	Visual embeddings from sampled video/image frames
Campaign brief + LLM structured extraction	Automated authorized brand/product profiling
Deterministic hard filters	Live social verification beyond POC / broad app review
Structured + semantic candidate ranking	Learned two-tower or ranking objective if labels support it
Reciprocal scoring	Advanced fairness optimization
Explainable top-K recommendations	Data-driven pricing recommendation
Minimal two-way proposal/interest flow	Rich messaging / CRM
Gold dataset + baseline/ablation evaluation	Cloud-scale feature store / AgentOps
Basic RAG for documents	Mutual reputational due diligence

7. Provisional UI/UX baseline (not frozen)

The attached wireframes are useful as a product baseline but the team has explicitly parked final UI decisions. Treat these routes and flows as provisional contracts, not pixel-locked requirements.

Mode	Primary navigation / flows
Shared	Single registration; activate Brand and/or Creator profile; switch professional context.
Brand	Wall, Discover, Campaigns, Collaborations, Messages, Profile; Campaign Studio; ranked creators; role-aware invitation.
Creator	Opportunities/discovery, campaign detail, proposals, collaborations, profile/rate card/social accounts.
AI interaction	Natural-language campaign brief → structured review; “Why this match?”; proposal draft; document policy Q&A.
Agreement	AI may draft readable terms from approved evidence; both parties approve/request changes; no self-executing agreement.

B. Software Architecture

8. Architecture decisions

Decision	Current choice	Reason
Repository	One GitHub monorepo	Five-person project; one PR can update frontend/backend/contracts together.
Backend style	FastAPI modular monolith	Python-native ML/LLM integration; avoids premature distributed systems.
Frontend	React + TypeScript + Vite	Engineering foundation is already aligned to Vite; earlier Next.js option is not required.
Database	PostgreSQL + pgvector	One source for relational truth and vectors; avoids separate vector DB.
ML	LR/RF baselines + XGBoost candidate	Appropriate for small/medium tabular data; supports controlled comparisons.
Embeddings	Sentence Transformers/BGE-class local encoder	Reproducible semantic retrieval at low cost.
LLM	Cloud provider through abstraction	Low operational burden; provider can be switched without redesign.
RAG	2-step retrieval first	Predictable, testable; agentic retrieval

Decision	Current choice	Reason
		only when justified.
Workflow	LangGraph or explicit state machine	Stateful orchestration, checkpoints and human approval.
Social APIs	Adapters with manual fallback	API access must not become critical path.
Deployment	Docker Compose + simple cloud	Reproducible demo without Kubernetes/microservice overhead.

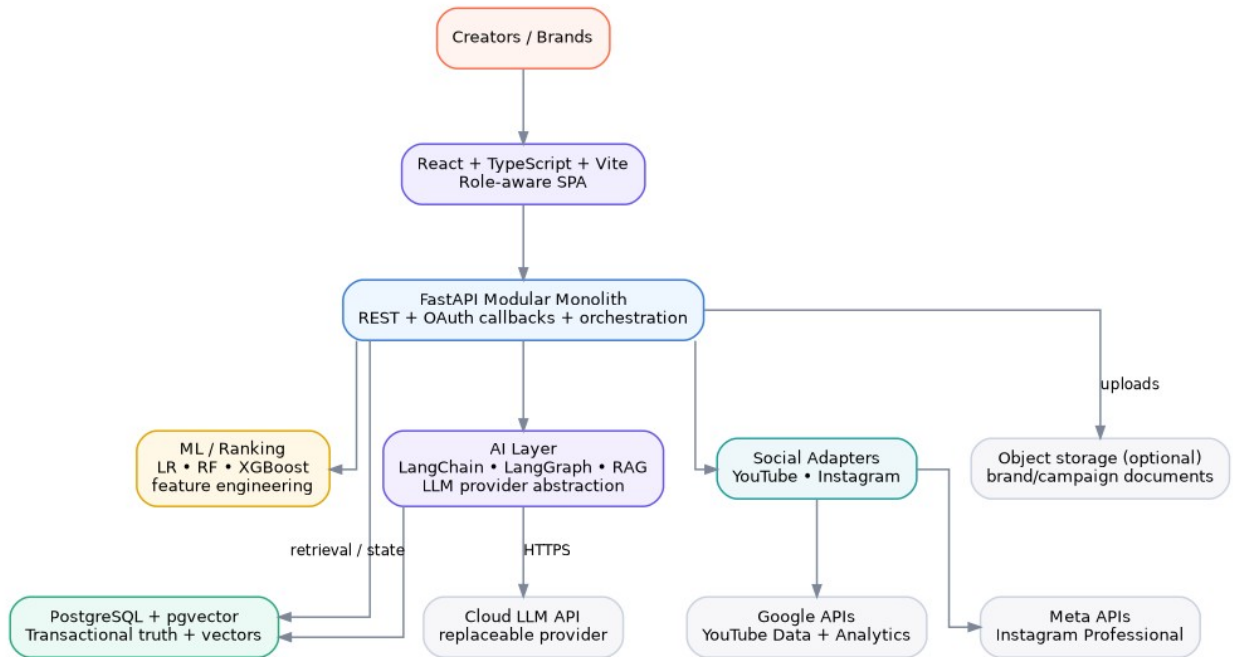


Figure 2. Current system architecture. External APIs and LLMs are adapters, not the center of the design.

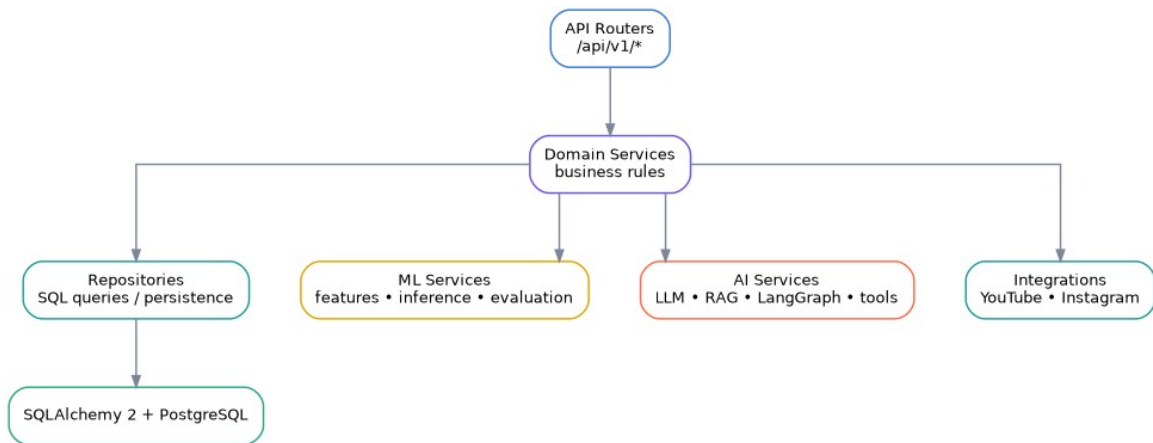


Figure 3. Backend modular-monolith separation of concerns.

9. Monorepo structure

brandbridge-ai/

```

├── frontend/
│   ├── src/{app,components,features,pages,services,hooks,types,utils,config,mocks}/
│   ├── tests/
│   └── Dockerfile
├── backend/
│   ├── app/
│   │   ├── api/v1/endpoints/
│   │   ├── core/
│   │   ├── database/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── repositories/
│   │   ├── services/
│   │   ├── ml/{features,training,inference,evaluation,artifacts}/
│   │   ├── ai/{llm,embeddings,rag,agents,tools,prompts}/
│   │   └── integrations/{youtube,instagram}/
│   ├── tests/{unit,integration}/
│   ├── alembic/
│   └── Dockerfile
├── notebooks/{data_analysis,feature_engineering,model_experiments,evaluation}/
├── data/{samples,synthetic,labels}/
├── docs/{architecture,api,ml,rag,diagrams,decisions}/
├── infra/{docker,deployment}/
├── scripts/{seed_data.py,train_model.py,sync_social.py}/
├── .github/{workflows,ISSUE_TEMPLATE,pull_request_template.md}
├── docker-compose.yml
├── .env.example
├── README.md
├── CONTRIBUTING.md
└── AGENTS.md

```

Separation rules

API schemas are separate from SQLAlchemy models. Route handlers call services; services call repositories/integrations/ML/AI modules. Notebooks never become production endpoints. AI modules do not own transactional facts.

C. Database Architecture and Schema

10. Database principles

- PostgreSQL is the source of truth for identity, profiles, campaigns, commercial terms, statuses and score components.
- pgvector stores document and semantic embeddings; vector rows always retain owner/version metadata.
- Social metrics are snapshots, not overwritten “current truth”, so freshness and growth can be measured.
- OAuth secrets/tokens are encrypted or stored through a secret reference. Never expose tokens to React or LLM prompts.
- Raw campaign brief and structured extracted brief are both retained for LLM extraction evaluation.
- Match scores store component values and model/feature versions so explanations and experiments are reproducible.
- Real/authorized versus simulated fields are explicitly tagged in research datasets.

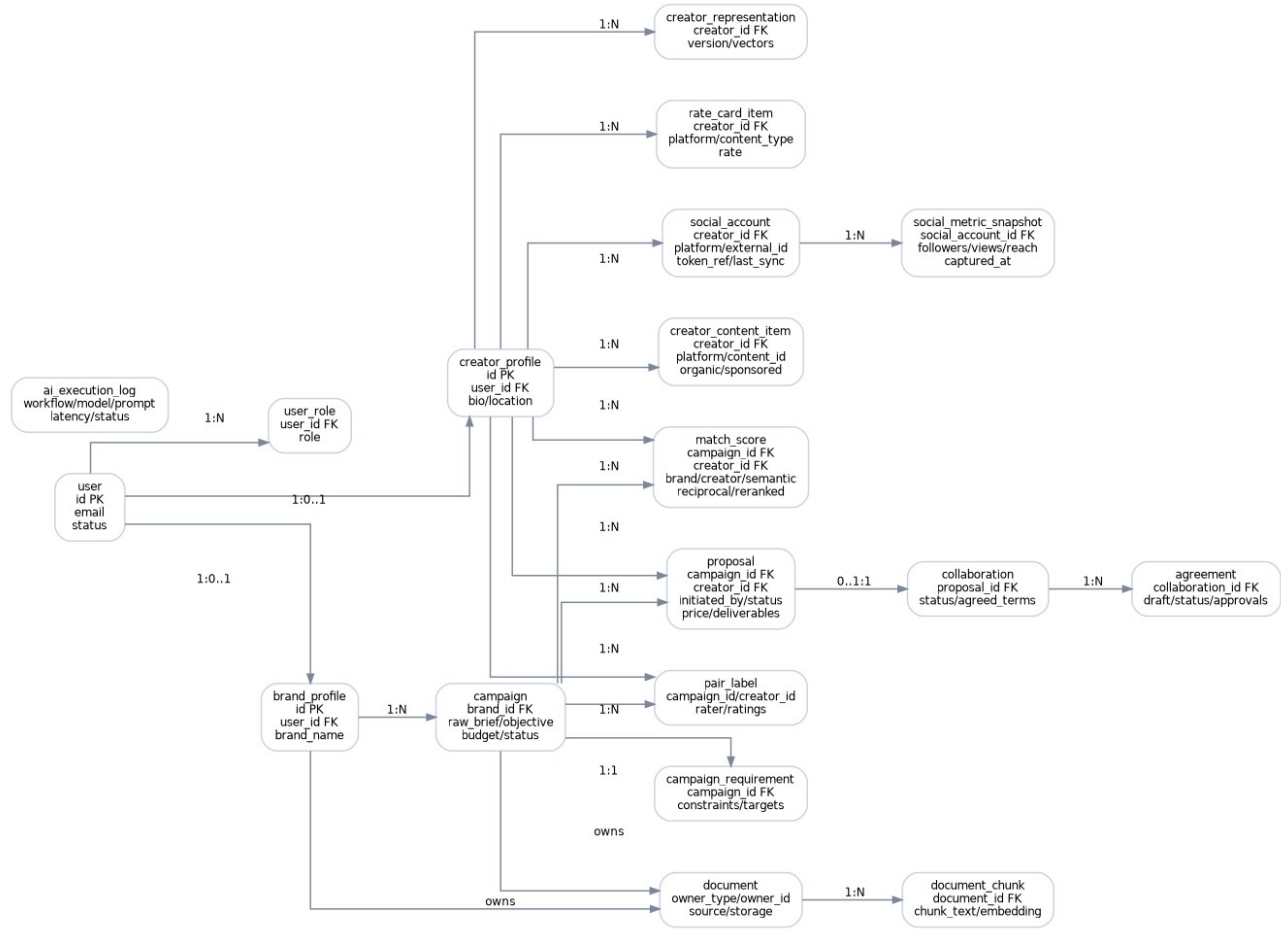


Figure 4. Core logical ERD. Some lookup and audit tables are omitted for readability.

11. Core schema - identity and profiles

Table	Important columns	Notes / indexes
users	id UUID PK; email CITEXT unique; password_hash; status; created_at; updated_at	Do not encode a single role here because one user can have both profiles.
user_roles	user_id FK; role ENUM(CREATOR, BRAND, ADMIN); created_at	Unique(user_id, role).
creator_profiles	id UUID PK; user_id unique FK; display_name; bio; location; country; languages JSONB; availability; travel_willingness; data_source flags	Index country/location; keep creator-entered business fields distinct from social-derived fields.
creator_preferences	creator_id FK; accepted_categories; excluded_categories; min_rate; preferred_models; values/exclusions JSONB	Used in creator-side suitability and deterministic exclusions.
brand_profiles	id UUID PK; user_id unique FK; brand_name; industry; description; website_url; primary_market; values/exclusions JSONB	Brand/product enrichment may later add evidence + freshness.

Table	Important columns	Notes / indexes
rate_card_items	id; creator_id; platform; content_type; rate_amount NUMERIC; currency; active; effective_from/to	Unique active item per creator/platform/content type.
creator_availability	id; creator_id; start_date; end_date; status; notes	Optional; can remain JSONB in MVP if simpler.

12. Social and creator-content schema

Table	Important columns	Purpose
social_accounts	id; creator_id; platform; external_account_id; username; account_type; verified; token_ciphertext/token_ref; refresh_token; granted_scopes; expires_at; last_synced_at	One row per connected provider account.
social_metric_snapshots	id; social_account_id; captured_at; followers; subscribers; total_views; avg_views; reach; interactions; engagement_rate; metrics_json	Time-series provider metrics; nullable fields expected.
creator_content_items	id; creator_id; social_account_id; external_content_id; media_type; caption/transcript; published_at; organic_or_sponsored; brand_entity; evidence; confidence; metrics_json	Separates organic themes from paid/sponsored history.
creator_representations	id; creator_id; representation_version; overall_embedding VECTOR; topic/style/format/value metadata; created_at	Versioned multi-view creator representation.
creator_content_clusters	id; representation_id; label; share; embedding VECTOR; metadata	Supports multi-vertical creator sub-profiles.

13. Campaign, constraints and knowledge schema

Table	Important columns	Purpose
campaigns	id; brand_id; name; raw_brief; objective; budget_min/max; currency; start/end; status; structured_brief JSONB; created_at	Matching query and lifecycle root.
campaign_requirements	campaign_id PK/FK; target_age_min/max; geography; languages; platforms; content_types; preferred_categories; mandatory_constraints JSONB; creator_discovery_mode; novelty_setting; min_transferable_fit	Explicit structured requirements confirmed by brand.
competitor_registry	id; parent_entity; brand; product_line; aliases; handles	Normalized competitor/conflict identity.
campaign_competitor_policies	id; campaign_id; competitor_id; association_type; lookback_days; exclusivity; action	Deterministic conflict rules; ambiguous evidence goes to review.

Table	Important columns	Purpose
documents	id; owner_type BRAND/CAMPAIGN; owner_id; document_type; filename; storage_uri; checksum; uploaded_by; created_at	Metadata for RAG source documents.
document_chunks	id; document_id; chunk_index; chunk_text; embedding VECTOR; metadata JSONB	RAG evidence store; index vector and document_id.

14. Matching, collaboration and research schema

Table	Important columns	Purpose
match_scores	id; campaign_id; creator_id; feature_version; model_version; eligibility_status; brand_score; creator_score; ml_score; semantic_score; reciprocal_score; reranked_score; components JSONB; evidence JSONB; created_at	Auditability, explainability and experiment comparison.
proposals	id; campaign_id; creator_id; initiated_by CREATOR/BRAND; proposed_amount; currency; deliverables JSONB; message; status; created_at; updated_at	One data model supports creator proposal and brand invitation semantics.
proposal_versions	id; proposal_id; version_no; amount; deliverables; message; created_by; created_at	Optional but recommended if counter-offer history is shown.
collaborations	id; accepted_proposal_id; status; agreed_amount; start/end; created_at	Created only after explicit human acceptance.
agreements	id; collaboration_id; draft_text; evidence_refs; status; brand_approved_at; creator_approved_at; version	AI-generated draft remains editable and non-self-executing.
messages	id; collaboration_id/proposal_id; sender_user_id; body; created_at	Simple internal messaging; realtime chat is not necessary.
pair_labels	id; campaign_id; creator_id; rater_id; brand_rating; creator_rating; rationale; label_source; created_at	Gold dataset / reciprocal evaluation.
collaboration_outcomes	id; proposal_id; accepted; completed; rating; outcome_metrics JSONB	Future evidence; do not fabricate sales/conversion labels.
ai_execution_logs	id; workflow_type; subject_type/id; provider; model; prompt_version; state_version; latency_ms; estimated_cost; status; error; created_at	Operational/research trace without storing secrets.

15. Required enums and state values

Enum	Values
user_role	CREATOR, BRAND, ADMIN

Enum	Values
campaign_status	DRAFT, READY, PUBLISHED, PAUSED, CLOSED, ARCHIVED
proposal_status	DRAFT, SUBMITTED, REVIEWING, COUNTER_OFFER, ACCEPTED, REJECTED, WITHDRAWN
collaboration_status	ACTIVE, IN_PRODUCTION, IN_REVIEW, COMPLETED, CANCELLED
platform	YOUTUBE, INSTAGRAM, MANUAL
content_type	REEL, STORY, POST, SHORT, VIDEO, OTHER
initiated_by	CREATOR, BRAND
data_provenance	CREATOR_PROVIDED, AUTHORIZED_API, PUBLIC_OBSERVABLE, AI_DERIVED, SYNTHETIC
discovery_mode	CONVENTIONAL, ADJACENT, FRESH_TO_VERTICAL, NO_PREFERENCE

D. API Architecture and Flow Contracts

16. API design rules

- Base prefix: /api/v1; use resource-oriented paths and explicit action endpoints only when state transition/AI operation is not simple CRUD.
- Pydantic v2 request/response schemas; never return SQLAlchemy models directly.
- All mutating endpoints authenticate the user and verify active role/profile ownership.
- Pagination uses limit/cursor or page/page_size consistently; pick one before implementation and document it.
- Errors return a common shape: code, message, details, correlation_id.
- Long AI/model operations should be synchronous only if fast enough; otherwise create an operation/run record and poll status.
- OpenAPI schema generated by FastAPI is the frontend contract; frontend types may be generated from it later.

Method	Path	Auth	Purpose
GET	/health	Public	Liveness
GET	/api/v1/health	Public	Versioned health
POST	/api/v1/auth/register	Public	Create identity + selected roles
POST	/api/v1/auth/login	Public	Login
POST	/api/v1/auth/refresh	Auth	Refresh session
GET	/api/v1/me	Auth	Identity + active profiles/roles

Method	Path	Auth	Purpose
GET/PATCH	/api/v1/creators/{creator_id}	Auth	Creator profile
GET	/api/v1/creators	Auth	Search/filter creator profiles
GET/PUT	/api/v1/creators/{id}/rate-card	Creator	Rate card
GET/PUT	/api/v1/creators/{id}/preferences	Creator	Preferences/exclusions
GET/PATCH	/api/v1/brands/{brand_id}	Auth	Brand profile
POST	/api/v1/campaigns	Brand	Create raw campaign
GET	/api/v1/campaigns	Auth	List campaigns
GET/PATCH	/api/v1/campaigns/{id}	Auth	Campaign detail/update
POST	/api/v1/campaigns/{id}/parse-brief	Brand	LLM extraction into reviewable structure
POST	/api/v1/campaigns/{id}/confirm-requirements	Brand	Persist confirmed requirements
POST	/api/v1/campaigns/{id}/publish	Brand	Publish after validation
POST	/api/v1/campaigns/{id}/recommendations/generate	Brand	Generate/persist top-K creator ranking
GET	/api/v1/campaigns/{id}/recommendations	Auth	Read persisted creator matches
GET	/api/v1/creators/{id}/recommendations	Creator	Recommended active campaigns
GET	/api/v1/matches/{campaign_id}/{creator_id}	Auth	Score components/evidence/explanation
POST	/api/v1/campaigns/{id}/proposals	Creator	Creator-initiated proposal
POST	/api/v1/campaigns/{id}/invitations	Brand	Brand-initiated invitation/proposal
GET	/api/v1/proposals	Auth	Inbox/outbox filters
GET	/api/v1/proposals/{id}	Auth	Proposal detail
POST	/api/v1/proposals/{id}/counter	Auth	Create counter version
POST	/api/v1/proposals/{id}/accept	Auth	Accept + create collaboration
POST	/api/v1/proposals/{id}/reject	Auth	Reject
GET	/api/v1/collaborations	Auth	List collaborations
GET/PATCH	/api/v1/collaborations/{id}	Auth	Track status
POST	/api/v1/collaborations/{id}/messages	Auth	Simple message/update

Method	Path	Auth	Purpose
POST	/api/v1/brands/{id}/documents	Brand	Upload knowledge asset
POST	/api/v1/campaigns/{id}/documents	Brand	Upload campaign document
POST	/api/v1/campaigns/{id}/knowledge/query	Auth	RAG question over authorized docs
GET	/api/v1/integrations/youtube/connect	Creator	Begin Google OAuth
GET	/api/v1/integrations/youtube/callback	Public callback	Exchange code/store account
GET	/api/v1/integrations/youtube/public-lookup	Auth	Public channel lookup by handle
POST	/api/v1/integrations/youtube/{account_id}/sync	Creator	Refresh normalized metrics
GET	/api/v1/integrations/instagram/connect	Creator	Begin Meta/Facebook Login OAuth
GET	/api/v1/integrations/instagram/callback	Public callback	Exchange/resolve Page + IG account
POST	/api/v1/integrations/instagram/{account_id}/sync	Creator	Refresh profile/media/insights allowed
POST	/api/v1/ai/proposals/draft	Auth	Generate editable proposal draft from stored evidence
GET	/api/v1/ai/runs/{id}	Auth/Admin	Inspect AI run status/trace metadata

17. Core API flow - campaign to ranked creators

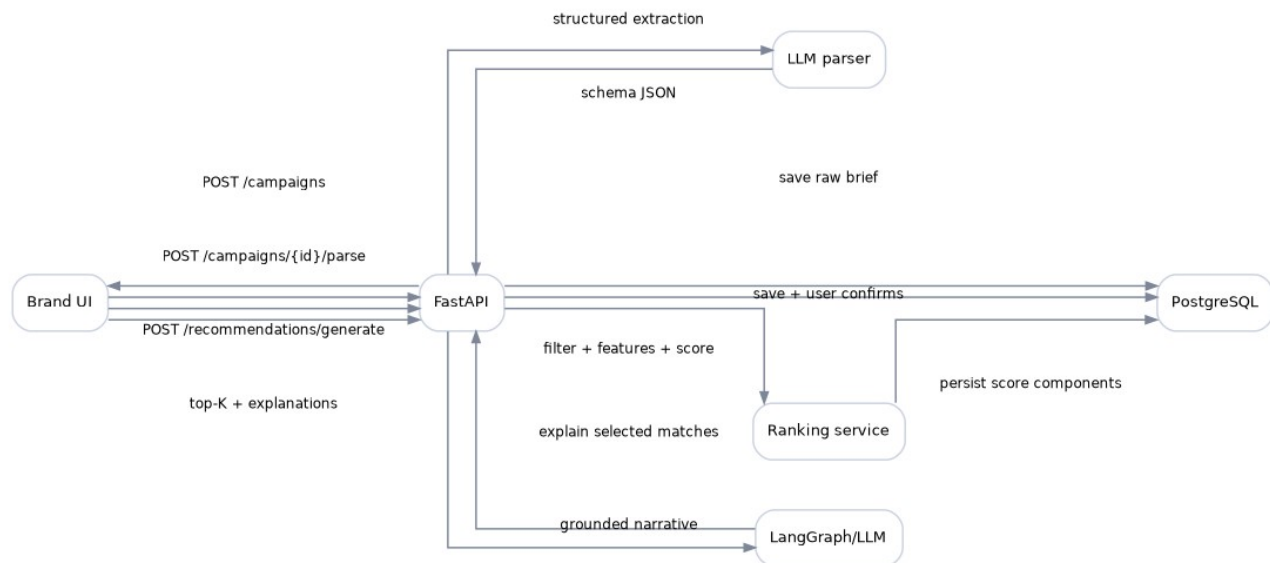


Figure 5. API sequence from campaign creation to persisted, explainable top-K recommendations.

1. Brand POSTs raw campaign brief. Backend stores raw text first.
2. Brand calls parse-brief. LLM returns schema-constrained JSON; backend validates and stores as a draft structured brief.
3. Brand edits/confirms requirements. Only confirmed requirements feed eligibility.
4. Recommendation generation applies hard filters, loads the creator representation, builds features, computes structured + semantic + creator-side scores, combines them, then reranks if enabled.
5. Backend persists every score component, version and evidence before returning top-K.
6. Explanation generation receives stored facts/components only; it cannot invent missing metrics.
7. Brand selects/invites a creator only after human review.

18. Example campaign parse contract

```
POST /api/v1/campaigns/{campaign_id}/parse-brief
{
  "brief": "High-protein millet snack. India. Fitness creators. Reels and Shorts. Budget INR 50K. No unsupported medical claims."
}

200 OK
{
  "campaign_id": "...",
  "parser_version": "campaign-parser-v1",
  "structured": {
    "category": "health_fitness",
    "country": ["IN"],
    "budget_max": 50000,
    "platforms": ["INSTAGRAM", "YOUTUBE"],
    "content_types": ["REEL", "SHORT"],
    "preferred_creator_categories": ["fitness", "nutrition"],
    "hard_constraints": ["NO_UNSUPPORTED_MEDICAL_CLAIMS"]
  },
  "requires_confirmation": true
}
```

19. Example match response

```
GET /api/v1/matches/{campaign_id}/{creator_id}
{
  "eligible": true,
  "final_score": 0.91,
  "scores": {
    "structured_ml": 0.89,
    "semantic": 0.94,
    "brand_side": 0.92,
    "creator_side": 0.87,
    "reciprocal": 0.90,
    "reranked": 0.91
  },
  "evidence": {
    "categories": ["fitness", "nutrition"],
    "rate_reel_inr": 7000,
    "engagement_rate": 0.058,
    "metrics_last_synced_at": "2026-09-18T10:00:00Z",
    "provenance": ["CREATOR_PROVIDED", "AUTHORIZED_API"]
  },
}
```

```
"explanation": "...grounded narrative..."
}
```

E. Social API Integration Architecture

20. Integration strategy

Social integrations are useful for verified identity and measurable creator signals, but the recommendation system searches the BrandBridge creator representation rather than querying Instagram/YouTube live for every recommendation. Cache normalized data and show freshness.

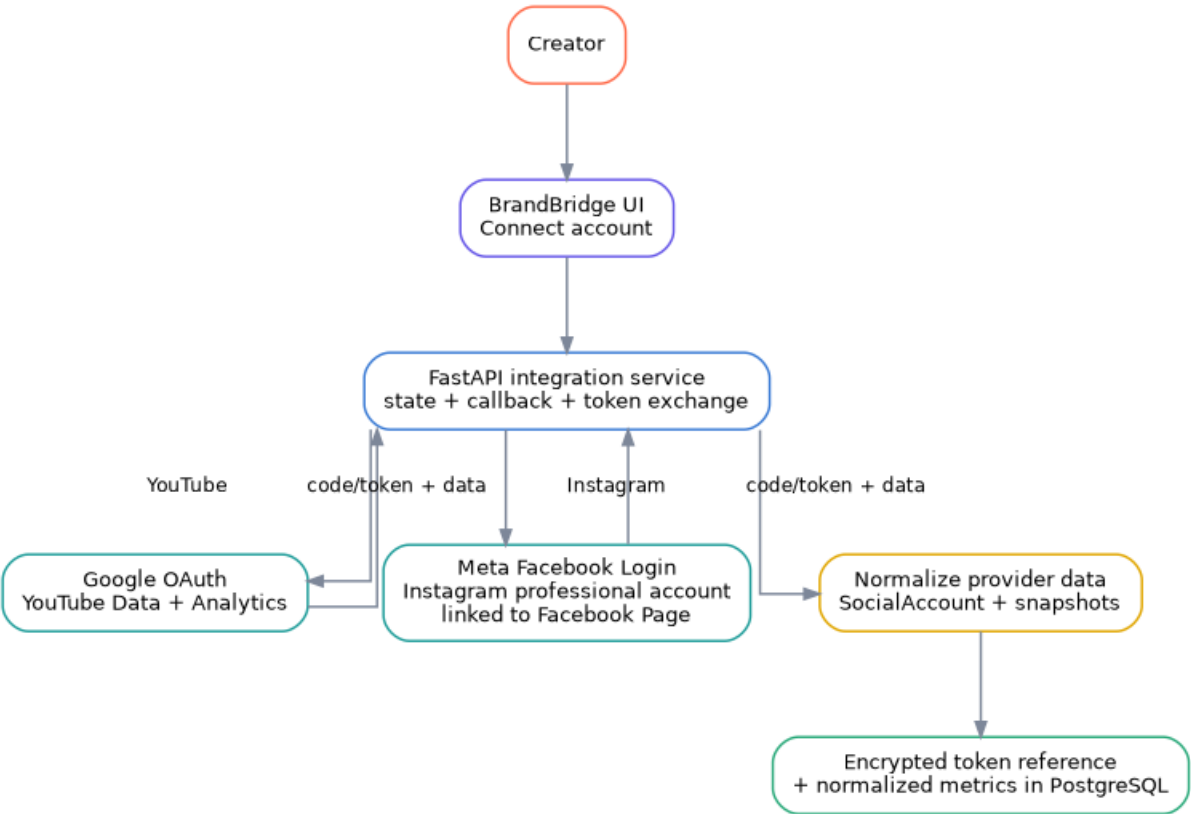


Figure 6. OAuth and normalization flow for YouTube and Instagram.

21. YouTube implementation path

Phase	Implementation
Public lookup	YouTube Data API channels.list using API key and handle/channel ID; map title, description, thumbnails, subscriber count, view count and video count into normalized fields.
Verification	Google OAuth Authorization Code flow through FastAPI callback; request minimum scopes such as youtube.readonly.
Analytics	For creator-authorized channel data, query YouTube Analytics with appropriate read-only scopes; persist snapshots, not raw tokens in frontend.

Phase	Implementation
Quota handling	Cache channel/metric data; log quota/API errors; do not make live calls during every ranking request.
Fallback	Creator can manually enter profile/rate data even when OAuth is unavailable.

Current Google notes

YouTube Data API requests require an API key or OAuth token. channels.list is a low-cost read call. YouTube Analytics is OAuth-authorized and reports are queried for the channel owner. Modern browser/web OAuth guidance favors Authorization Code + PKCE/server-side exchange rather than insecure implicit tokens.

22. Instagram implementation path - current team setup

Step	Implementation decision
Account prerequisite	Use Instagram Professional (Creator/Business) account linked to a Facebook Page for the Facebook Login API path currently configured by the team.
Meta app	Brand Bridge AI app; Instagram API use case; Facebook Login setup for insights-oriented flow.
POC first	Use Graph API Explorer / test role to confirm /me/accounts returns Page + instagram_business_account before production OAuth coding.
Production OAuth	React calls FastAPI /connect; backend creates state; redirects to Meta; callback exchanges code; backend resolves Page and linked Instagram professional ID.
Data sync	Fetch only permitted profile/media/insight fields; map them to SocialAccount, SocialMetricSnapshot and CreatorContentItem.
Security	Never put app secret/page/user access token in React, screenshots, logs or LLM prompts.
Resilience	If Meta access/app review changes, BrandBridge still works with manual/seed profiles and cached last-known metrics.

Important API constraint

Treat Meta permissions and exact available insight metrics as versioned external contracts. Revalidate them in the Meta developer dashboard immediately before implementation and submission; do not hard-code assumptions from old tutorials.

23. Normalized social data contract

```
{
  "platform": "INSTAGRAM",
  "external_account_id": "...",
  "handle": "@creator",
  "display_name": "Creator Name",
  "verified_by_oauth": true,
  "metrics": {
    "followers": 84732,
    "subscribers": null,
    "average_views": 27000,
    "reach": 180000,
    "engagement_rate": 0.058
  },
  "audience": {
    "primary_country": "IN",
    "age_bands": null
  },
  "captured_at": "...",
  "provenance": "AUTHORIZED_API"
}
```

F. Intelligence and Research Architecture

24. Creator representation and data pipeline

The creator is not a single category or one opaque vector. Build a structured research object from creator-provided business data, authorized social data and AI-derived content representations.

Evidence source	Examples	Use
Creator-provided	Location, languages, categories, rates, availability, accepted/excluded categories, travel, past collaborations, competitor restrictions	Hard constraints + creator-side suitability.
Authorized social	Profile, recent posts/reels/videos, captions, media metadata, permitted engagement/insight metrics	Verified scale/performance and content corpus.
AI-derived	Text embeddings, content clusters, topics/style/format/value views; optional visual embeddings/transcripts	Semantic retrieval and cross-vertical matching.

Dataset scope reconciliation

The consolidated V3 plan starts with roughly 100-300 creator profiles, while the dedicated creator strategy recommends 300-500 and a working target near 400. Use a phased target: 150-200 minimum experiment, 300-400 target if acquisition is practical, plus 30-50 consenting creators as the gold-standard validation set.

25. Campaign-to-match intelligence pipeline



Figure 7. Recommended retrieval → ranking → reciprocal scoring → grounded explanation pipeline.

8. Parse: LLM extracts candidate requirements from free text into a strict schema.
9. Confirm: Brand reviews/edits parsed fields. Unconfirmed LLM guesses never become hard constraints.
10. Eligibility: Deterministic rules remove impossible pairs (platform, geography, budget, exclusions, competitor restrictions).
11. Retrieval: Semantic vector search finds conceptually relevant creators from BrandBridge representations, not live social search.
12. Features: Build one creator-campaign row from structured profile, social snapshots, semantic similarities and missingness indicators.
13. Ranking: Compare simple baselines first; evaluate XGBoost as the stronger structured model.
14. Reciprocity: Compute brand-side and creator-side suitability separately, then test combination functions.
15. Objective awareness: Campaign objective can influence weights, but weights must be validated rather than assumed.
16. Fairness/diversity: Re-rank similarly qualified candidates with a minimum relevance safeguard.
17. Explanation: Persist evidence and components; LLM converts those facts into readable reasons only.

26. Machine-learning design

Model / method	Role	Primary evaluation
Popularity / follower baseline	Simple comparator; demonstrates why follower count alone is weak	Precision@K / NDCG@K
Rule-based structured baseline	Eligibility + hand-coded feature score	Ranking metrics + error analysis
Logistic Regression	Interpretable supervised baseline	F1/ROC-AUC where binary; ranking after score sort
Random Forest	Non-linear tabular baseline	Ranking metrics
XGBoost classifier/regressor	Primary structured candidate if it improves held-out ranking	Precision@5, Recall@5, NDCG@5, MAP
XGBoost ranker	Optional if pairwise/listwise labels are sufficient	NDCG/MAP
Semantic-only	Embeddings + cosine similarity	Ablation baseline
Hybrid reciprocal	Structured + semantic + two-sided suitability	Mutual NDCG@K, mismatch rate, joint utility

27. Candidate features

Feature family	Examples
Profile fit	category similarity; creator niche; content-format compatibility
Audience fit	country/language overlap; target-age overlap when verified

Feature family	Examples
Channel fit	platform availability; deliverable support
Commercial fit	rate vs budget; min-rate; compensation-model compatibility
Performance	engagement; average/median views; consistency; follower/subscriber scale; growth
Semantic fit	creator overall + cluster vectors vs campaign/product text
Preference / reciprocity	accepted/excluded categories; values; travel; availability; creator preferences
Trust/history	reliability; historical collaboration where real; sponsored-category history
Missingness	has_verified_audience; has_api_metrics; metric_age_days

28. LangChain / RAG responsibilities

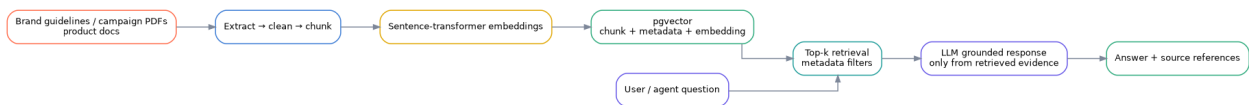


Figure 8. Two-step RAG pipeline. Structured facts remain in SQL.

Component	Allowed responsibility	Do not use it for
LangChain	Provider adapters, prompts, structured outputs, retrievers, tool wrappers	Core business rules or replacing typed services.
RAG	Brand guidelines, campaign briefs, product docs, creator portfolio documents	Budget, proposal status, user IDs, rate cards or invented match scores.
LLM	Campaign parsing, grounded explanation, proposal/agreement draft, document Q&A	Numeric scoring, hard eligibility, database search that can be deterministic.
Embeddings	Semantic retrieval and content/campaign similarity	Claiming causal campaign performance.

29. LangGraph orchestration

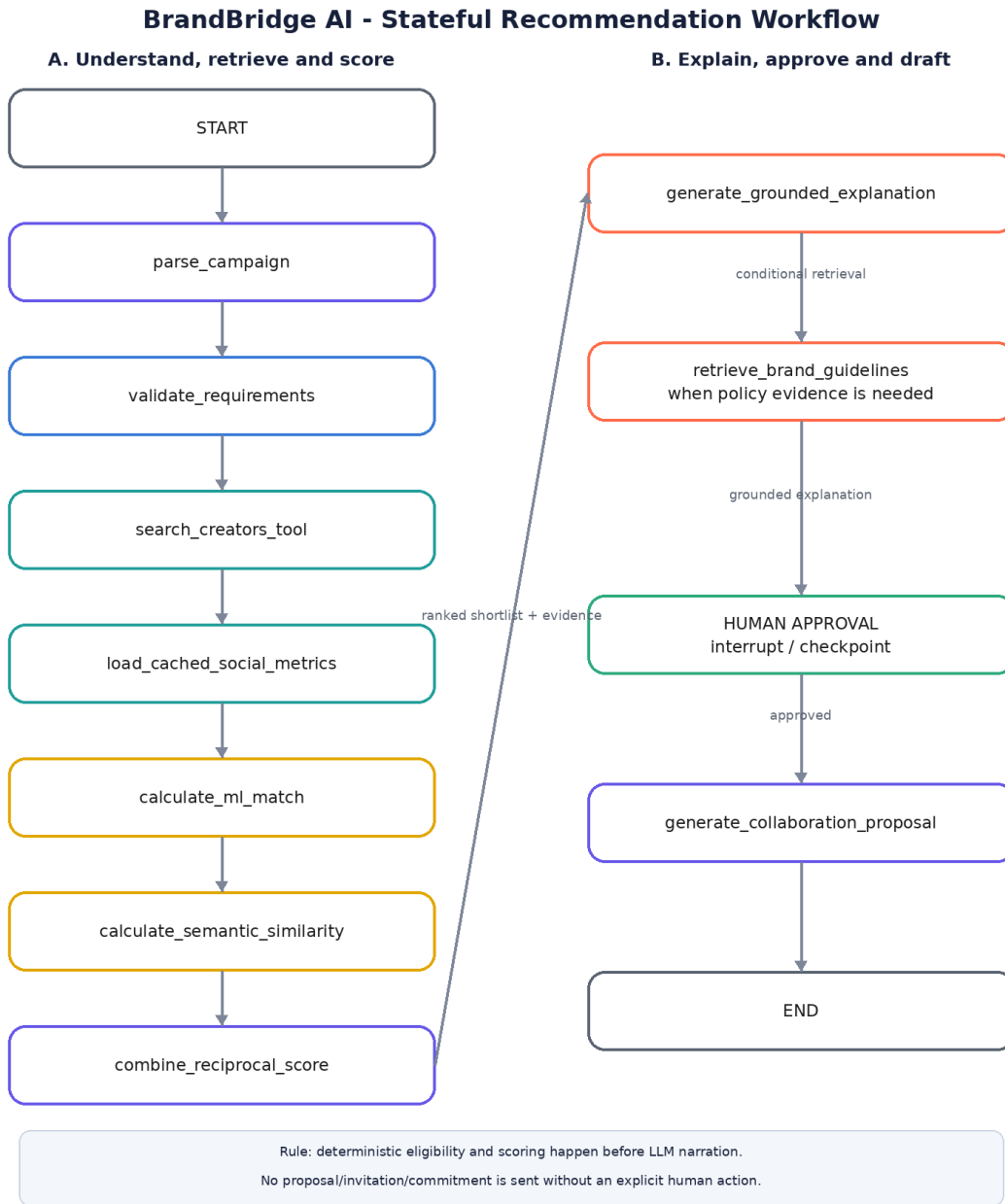


Figure 9. LangGraph workflow: deterministic tools first, evidence-grounded generation, explicit human approval.

29.1 Suggested graph state

CampaignMatchState

- request_id
- actor_user_id / active_role
- campaign_id
- raw_brief
- confirmed_requirements
- eligible_creator_ids
- social_metric_snapshot_ids
- feature_version
- model_version
- structured_scores

- semantic_scores
- creator_side_scores
- reciprocal_scores
- retrieved_document_chunk_ids
- explanation_objects
- selected_creator_ids
- approval_status
- proposal_draft_id
- errors / warnings

29.2 Reusable tools/skills

Tool	Deterministic / AI	Contract
search_creators	Deterministic	SQL hard filters and candidate lookup
get_creator_profile	Deterministic	Fetch normalized creator representation
get_youtube_statistics	Integration	Load cached/current authorized provider data
get_instagram_statistics	Integration	Load cached/current authorized provider data
calculate_ml_match	ML deterministic inference	Return structured score + feature version
calculate_semantic_similarity	Embedding	Return semantic score + representation version
retrieve_brand_guidelines	RAG retrieval	Return top-k chunks + source metadata
generate_match_explanation	LLM grounded	Input is score/evidence object only
generate_collaboration_proposal	LLM grounded	Draft from approved campaign + commercial terms

G. Collaboration Workflow and Human Control

30. Proposal/invitation state machine

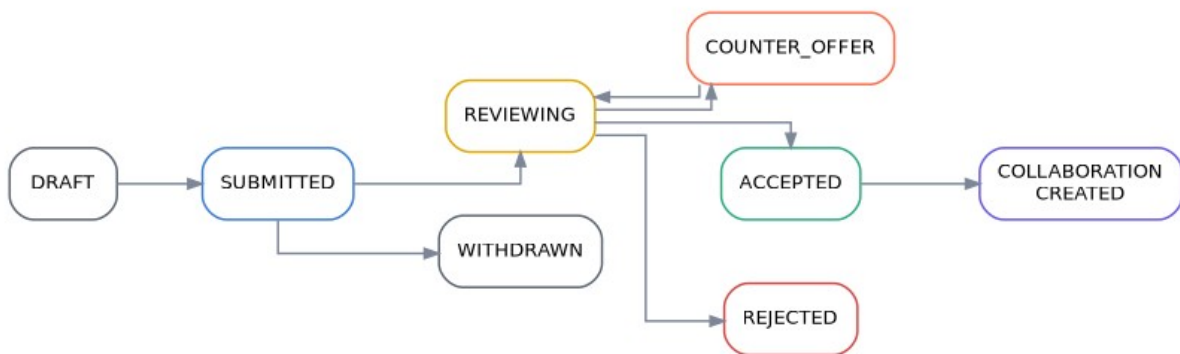


Figure 10. Creator proposal and Brand invitation converge on one proposal state model.

- Creator starts collaboration → proposal row initiated_by=CREATOR.
- Brand starts collaboration → invitation endpoint creates proposal row initiated_by=BRAND.

- Counter-offer creates a new proposal version; never overwrite commercial history.
- Only explicit ACCEPT creates a collaboration row.
- AI can draft messages/agreements but user must review/edit and approve before send/accept.
- Agreement text should retain evidence references and versioned approvals if implemented.

H. Dataset, Evaluation and Scientific Method

31. Data strategy

Dataset component	Working target	Use
Creator profiles	150-200 minimum; 300-400 target if feasible	Structured/semantic features; retrieval/ranking.
Consenting creator gold set	30-50	Real rates/preferences/exclusions/audience details for validation.
Campaign briefs	~160 labelled briefs: ~120 core verticals + ~40 unseen verticals	Parser evaluation and generalization.
Pair labels	Multiple raters on sampled creator-campaign pairs	Ranking ground truth.
Creator-side preference labels	Explicit rubric/personas or creator survey	Reciprocal evaluation.
RAG documents/questions	Small controlled guideline corpus + 30-50 questions	Retrieval hit rate and groundedness.
Interaction outcomes	Collect from prototype if available	Future labels only; do not assume at project start.

Label rubric

Suggested ordinal relevance: Excellent=3, Good=2, Borderline=1, Poor=0. A mutually relevant pair may require both brand-side and creator-side rating $\geq$ a predeclared threshold. Use multiple raters, measure agreement and adjudicate disagreements.

32. Experiment sequence

18. Popularity/follower baseline.
19. Rule-based structured baseline.
20. Semantic-only ranking.
21. One-sided hybrid ranking (structured + semantic).
22. Reciprocal hybrid ranking.
23. Reciprocal ranking + fairness/diversity reranking.
24. Optional controlled ablations: multi-vector creator representation; visual features; discovery mode; competitor policy.

33. Evaluation metrics

Component	Metrics / tests
Top-K ranking	Precision@5, Recall@5, NDCG@5, MAP
Reciprocity	Mutual Precision@K / Mutual NDCG@K, one-sided mismatch rate, joint utility
Cold-start/exploration	Human relevance, NDCG@K, minimum-fit pass rate, fresh-to-vertical share, category coverage
Competitor rules	Rule accuracy, false exclusions, review rate
Campaign extraction	Per-field accuracy, schema-valid rate, missing-field handling
RAG	Top-k retrieval hit rate, evidence correctness, groundedness
Explanation/proposal	Factual consistency, usefulness rubric, edit distance/user rating
Agent workflow	Correct tool sequence, pause/resume, no unauthorized action, recovery
System	Latency, failure rate, Docker startup, API tests, security checks

I. Security, Privacy and Responsible AI

34. Security requirements

Risk	Required control
OAuth token leakage	Server-side token storage; encrypted field/secret reference; redact logs; never send tokens to LLM.
Excessive permissions	Request minimum provider scopes; document rationale; revoke/disconnect path.
PII in prompts	Send only task-required fields; prefer IDs/aggregated metrics.
Hallucinated metrics	LLM only receives stored metrics/evidence; missing data is explicit.
Prompt injection in uploaded docs	Treat documents/social text as untrusted; isolate retrieval evidence; never execute embedded instructions.
Stale provider data	Store captured_at/last_synced_at and show freshness; do not present stale data as current.
Sensitive-trait inference	Do not infer religion, health, politics or other sensitive traits from creator content.

Risk	Required control
Unauthorized scraping	Use public/authorized data and platform terms; social scraping is not a system dependency.
Commercial autonomy	Human approval for outreach, counters, agreement and acceptance.

J. Docker, Cloud, CI/CD and Observability

35. Local development topology

services:

frontend:

build: ./frontend

ports: ["5173:5173"]

environment:

VITE_API_BASE_URL: http://localhost:8000

backend:

build: ./backend

ports: ["8000:8000"]

env_file: .env

depends_on: [db]

db:

image: pgvector/pgvector:pg16

ports: ["5432:5432"]

volumes:

- brandbridge_db:/var/lib/postgresql/data

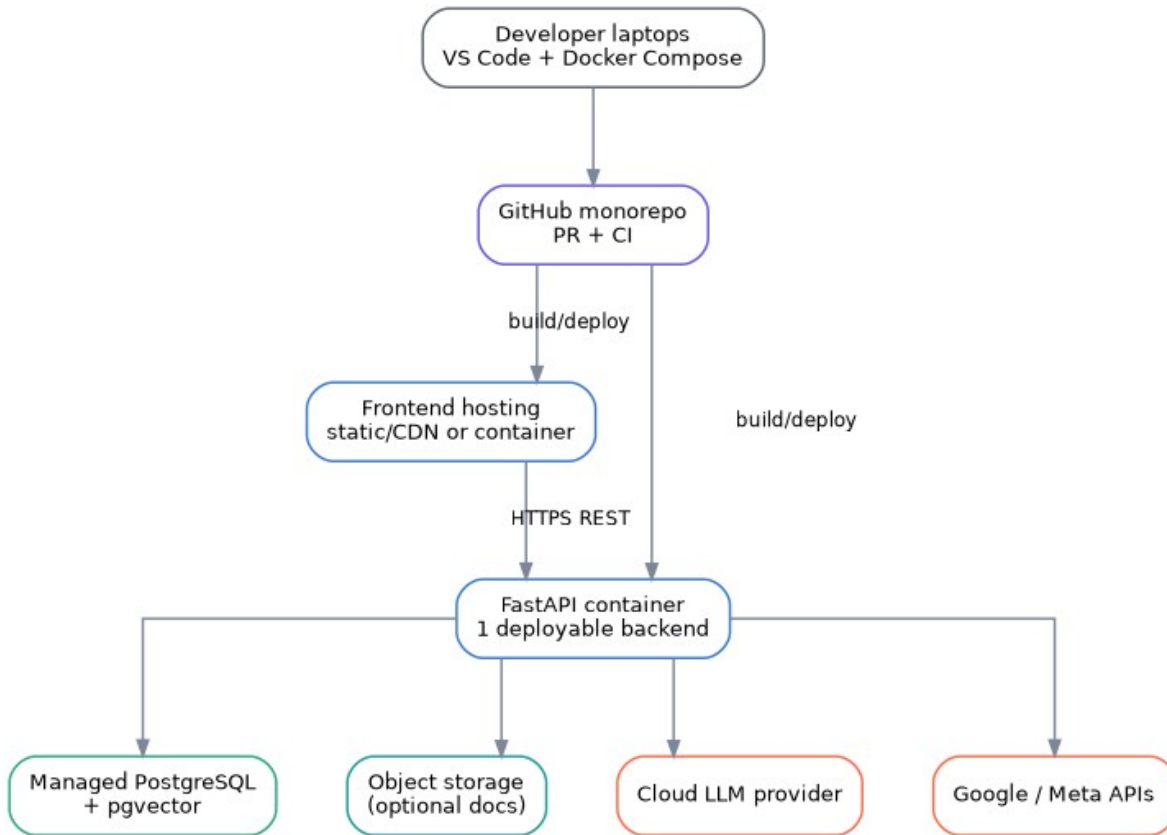


Figure 11. Local-to-cloud deployment model. Keep deployment simple and reproducible.

36. Environment variables

```

APP_ENV=
APP_NAME=BrandBridge AI
API_V1_PREFIX=/api/v1
DATABASE_URL=
JWT_SECRET=
JWT_ALGORITHM=HS256
CORS_ORIGINS=
LLM_PROVIDER=
LLM_API_KEY=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
YOUTUBE_API_KEY=
META_APP_ID=
META_APP_SECRET=
OAUTH_TOKEN_ENCRYPTION_KEY=
DOCUMENT_STORAGE_PATH=
  
```

37. CI pipeline

Trigger	Frontend	Backend	Additional
Pull request	npm ci → lint → test/build	install → Ruff → pytest	Optional OpenAPI diff + Docker build
Push to main	same	same	Build tagged images; deploy

Trigger	Frontend	Backend	Additional
			only when team is ready
Release tag	production build	production build	Simple cloud deployment + smoke tests

38. Observability

- Application logs with correlation_id/request_id; never log OAuth secrets or full access tokens.
- AI runs log provider/model/prompt version, latency, token/cost estimate when available, status and error category.
- Recommendation runs log feature/model/representation versions and candidate counts.
- Social sync logs provider account, request class, success/failure, freshness and rate-limit/quota errors.
- Prometheus/Grafana are stretch; structured logs + lightweight dashboard/report are enough for MVP.

K. Team Execution Plan

39. Git and code governance

- Protect main; no direct commits.
- Branches: feature/<scope>, fix/<issue>, spike/<experiment> for disposable API/model POCs.
- Issue → branch → implementation → tests → PR → one teammate review → merge.
- Every PR states description, related issue, testing, screenshots/API evidence, risks and documentation impact.
- Keep ADRs under docs/decisions and update the technical guide/README when architectural decisions change.

ADR	Decision
001-monorepo	Single repository
002-modular-monolith	One FastAPI deployable backend
003-fastapi-backend	Python/FastAPI service layer
004-postgresql-pgvector	Relational truth + vectors in PostgreSQL
005-hybrid-recommendation	Deterministic eligibility + structured/semantic ranking
006-langgraph-orchestration	LangGraph for stateful AI workflow after core functions work
007-youtube-first-social-integration	YouTube public/OAuth first; Instagram professional integration in parallel POC
008-human-in-the-loop	No autonomous commercial actions
009-rag-boundary	RAG only for unstructured knowledge
010-explainability-storage	Persist components/evidence before generating narrative

40. Five-person workstreams

Workstream	Primary outputs	Suggested interfaces with others
Product / Frontend	Role journeys, profile/campaign forms, ranked results, explanations, proposal/collaboration screens	Consumes OpenAPI; owns UX acceptance criteria; does not implement ranking logic.
Backend / Data	FastAPI, SQLAlchemy/Alembic, schema, auth hooks, hard rules, proposal states, score persistence	Publishes OpenAPI + DB migrations; integrates ML/AI/social adapters.
ML / Evaluation	Labels, features, LR/RF/XGBoost, reciprocal experiments, metrics, fairness/cold-start	Receives normalized creator/campaign data; returns typed score object.
Semantic / GenAI / RAG	Embeddings, retrieval, campaign extraction, explanation, RAG, LangGraph, evaluation	Uses typed tools/contracts; never owns DB truth or hard constraints.
Integration / QA / Documentation	YouTube/Instagram POCs, scenario tests, experiment log, ethics/privacy, deployment, report/demo	Maintains integration fixtures/fallbacks and end-to-end acceptance suite.

Non-technical member contribution

Dataset labelling, campaign/persona design, creator/brand research, consented data collection, test scenarios, evaluation rubrics, UAT, experiment log and presentation are substantive capstone contributions.

41. Milestones and technical path

Period	Definition of done
Now / Week 0	Freeze this architecture baseline; finish YouTube + Instagram POCs; create monorepo foundation.
Week 1	PostgreSQL schema + Alembic; identity/dual profile; seed script; health/auth skeleton.
Week 2	Creator/brand profile + rate card + campaign CRUD; provisional frontend forms.
Week 3	Hard eligibility rules; campaign structured schema; label rubric; baseline dataset.
Week 4	Semantic creator representations + candidate retrieval; social normalized snapshots.
Week 5	Baseline LR/RF + XGBoost experiments; evaluation split; persist match components.
Week 6	Reciprocal brand/creator scoring; creator-side campaign recommendation.
Week 7	LLM campaign parser with structured validation; extraction test set.
Week 8	RAG ingestion/retrieval for brand/campaign docs; grounding

Period	Definition of done
	evaluation.
Week 9	LangGraph orchestration + human approval; explanation/proposal draft.
Week 10	Proposal/collaboration UI/API; integration tests; fairness/cold-start experiment.
Week 11	Docker/cloud hardening; API failure fallbacks; latency/cost/error logs.
Week 12	Ablation results, final demo data, report/deck, contribution log, buffer.

42. Definition of done by feature

Area	Done means
Backend feature	Schema/migration if required; service/repository separation; Pydantic contract; unit/integration tests; OpenAPI updated.
Frontend feature	Route/page works against typed API/mock; loading/error/empty states; role authorization; screenshot in PR.
ML experiment	Dataset/model/feature versions recorded; held-out evaluation; metrics + error analysis; reproducible script.
AI/RAG feature	Structured inputs/outputs; prompt/retriever version; grounded test set; failure behavior; no uncontrolled actions.
Social integration	OAuth/test flow works; secrets safe; normalized mapping tested; cached fallback; disconnect/expiry behavior documented.
Release	Docker build passes; CI green; seed/demo mode works; known limitations documented.

L. Risks, Open Questions and Decision Log

43. Main risks and mitigations

Risk	Mitigation
Insufficient historical collaboration outcomes	Use transparent public/authorized + synthetic business variables and multi-rater gold labels; narrow claims.
Creator-side preference poorly defined	Design explicit preference fields and reciprocal rating rubric early; user interviews if possible.
Social API access/app review blocks demo	Keep manual profiles and cached/seed metrics; POC remains

Risk	Mitigation
	non-blocking.
LLM hallucinated explanations	Only pass stored score components/evidence; schema outputs; RAG for policies.
Metric leakage / overfit	Split by creator/campaign; deduplicate near-identical examples; hold out final test set.
Fairness becomes vague	Predeclare measurable exposure objective and minimum relevance threshold; report trade-offs.
Team overbuilds marketplace	Freeze core MVP; treat advanced messaging/payments/pricing as stretch.
Too many AI components	Academic core remains recommendation experiment; RAG/agent support, not distract.

44. Open questions that require team decision

25. How exactly will creator-side preferences be captured: explicit profile fields, personas, surveys, or a mixture?
26. What fairness objective will be defended: follower-size exposure, opportunity for qualified micro/nano creators, category/geography coverage, or another metric?
27. How much creator data can be legally/practically obtained this semester, and what percentage will be simulated?
28. Will optional visual embeddings add measurable value beyond text + structured features within time?
29. Which campaign objectives are in the MVP and how will objective-specific weights be validated?
30. What constitutes the gold label: brand relevance, two independent side ratings, or a joint rubric?
31. Which LLM/RAG features are essential to the final demo versus optional?
32. How much real user evaluation can be conducted with creators/brand managers or proxy raters?
33. Which brand/product enrichment sources are supported, and how will provenance/freshness be displayed?

45. Accepted design decisions

- Keep the problem two-sided and treat academic core as reciprocal recommendation/ranking.
- Use retrieval → ranking; a trained two-tower model is not required.
- Hard constraints are deterministic and separate from AI scoring.
- Creators can have multiple interpretable content views/clusters rather than one category.
- Organic content interest is separate from sponsored-category/competitor history.
- Brand-side and creator-side suitability are computed separately before reciprocal combination.
- Campaign objective may influence scoring but must be validated.
- Cold-start and fair exposure are studied without equating diversity with fairness.
- Explanations are auditable and grounded in stored facts/scores.
- RAG is for documents; SQL is for structured facts.
- External social APIs are optional adapters; manual fallback is mandatory.
- Human approval is required for commercial decisions.

Appendix A. Recommended first implementation issues

34. Create monorepo skeleton + Docker Compose + health endpoints.

35. Define SQLAlchemy/Alembic identity/profile/campaign/proposal core schema.
36. Implement seed data for Brand, Creator, campaign and rate card.
37. Finish YouTube public lookup + OAuth POC and normalize response fixture.
38. Finish Instagram Facebook-login Graph POC and normalize response fixture.
39. Implement creator/brand profile CRUD.
40. Implement campaign raw brief CRUD + confirmed structured requirement schema.
41. Implement deterministic hard-filter service with unit tests.
42. Create creator representation builder from manual/seed content.
43. Create label rubric + first labelled pair dataset.
44. Implement baseline semantic retrieval and structured feature pipeline.
45. Implement LR/RF/XGBoost evaluation notebook/script.
46. Persist match scores/components and expose GET match explanation object.
47. Implement reciprocal creator-side campaign recommendation.
48. Implement LLM structured campaign parser with test set.
49. Implement document upload/chunk/embed/retrieve RAG path.
50. Implement LangGraph workflow after individual tools are tested.
51. Implement proposal/invitation state machine and collaboration creation.
52. Add CI, Docker smoke test and final demo seed mode.

Appendix B. References and revalidation notes

Reference	URL	Use
Google YouTube Data API Overview	https://developers.google.com/youtube/v3/getting-started	Quota and API-key/OAuth requirements; recheck quota before demo.
YouTube channels.list	https://developers.google.com/youtube/v3/docs/channels/list	Public/authenticated channel lookup.
YouTube Analytics API Reference	https://developers.google.com/youtube/analytics/reference	OAuth-authorized reports.
YouTube Analytics Channel Reports	https://developers.google.com/youtube/analytics/channel_reports	Channel metrics/dimensions and ownership requirements.
Google OAuth guidance	https://developers.google.com/youtube/reporting/guides/authorization	OAuth scopes and verification considerations.
LangGraph overview	https://docs.langchain.com/oss/python/langgraph/overview	Stateful orchestration and human-in-the-loop concepts.
LangChain retrieval	https://docs.langchain.com/oss/python/langchain/retrieval	Loaders/splitters/embeddings/vector stores/retrievers.
Meta Instagram API workspace	https://www.postman.com/meta/instagram/documentation/6yqw8pt/instagram-api	Professional-account API reference; revalidate exact permissions/metrics in current dashboard.

Revalidation rule

Social APIs, OAuth scopes, quotas and LLM provider pricing change. Revalidate external API details immediately before final implementation/demo. Keep the internal normalized contract stable so provider changes do not ripple through ML/UI code.

Appendix C. Final demo acceptance scenario

53. Brand logs in and creates a campaign from natural language.
54. LLM returns structured requirements; Brand reviews and confirms them.
55. Backend applies hard constraints and retrieves eligible creators.
56. System shows top creators with structured, semantic, reciprocal and freshness evidence.
57. Brand opens “Why this creator?” and sees grounded explanation based on stored components.
58. Brand asks a policy question against an uploaded guideline document; RAG returns evidence-backed answer.
59. Brand initiates invitation; AI may draft but user approves the message.
60. Creator switches into Creator mode, sees invitation/recommended campaign, and submits/counters a proposal.
61. Explicit acceptance creates collaboration; status appears in workspace.
62. Team shows evaluation results (baseline vs semantic/hybrid/reciprocal), Docker/cloud architecture, limitations and future work.

Capstone presentation message

The marketplace is the product demonstration; the academic contribution is the explainable reciprocal recommendation experiment and controlled, grounded AI workflow around it.